1 Evaluation Methodology

This chapter details the methodology used to evaluate the Agentic Project Management (APM) framework. To assess the framework in a realistic setting, we conducted a comparative case study involving human participants. The study compares APM, a multi-agent framework that distributes work across specialised agents, against GitHub Spec-kit, a single-agent toolkit that guides one AI agent through a structured specification-driven workflow. Both frameworks are described in Section 1.2. The study was conducted in May 2026 on the Claude Code command-line interface; pricing, model, and command-set statements in this chapter refer to that snapshot.

The evaluation measures three distinct dimensions:

- Economic Efficiency (dollar cost of a session, computed against the platform's per-token rate card)
- Process Efficiency (human cognitive load and interaction burden across a session)
- Functional Efficacy (ability to complete non-trivial tasks)

1.1 Research Objectives

The primary objective of this research is to determine whether the structural overhead introduced by APM, in the form of multi-agent orchestration and the explicit Planning and Implementation phases, produces measurable improvements over a single-agent workflow. This evaluation contributes a cost perspective alongside the process and functional dimensions.

We define the following Research Questions (RQs):

- **RQ1 (Economic Efficiency):**
  - Does APM's multi-agent coordination produce a measurably different dollar cost per session than Spec-kit's single-agent workflow?
  - Where in each framework's workflow does session cost concentrate between the planning phase and the implementation phase, and does that concentration pattern differ between frameworks?
  - Does cumulative session cost comparison between the two frameworks shift, when using models with different context-window capacities and capabilities?
- **RQ2 (Process Efficiency):**
  - Does APM's structure reduce the amount of manual effort the user must invest during execution, particularly interventions, corrections, and clarifications?
  - If a difference in user effort is observed, is it significant enough to justify the structural overhead APM introduces?
  - Given that the user remains in the loop as a message carrier, can the agents coordinate effectively enough that the user's role can be limited to shuttling messages rather than correcting or redirecting agent behaviour?

- **RQ3 (Functional Efficacy):**
  - Can APM successfully deliver complex, non-trivial features in scenarios where single-agent workflows are prone to failure due to context window exhaustion, loss of coherence across long sessions, or inability to coordinate work across multiple concerns?
  - How does the performance of APM compare to the performance of a single-agent workflow?

1.2 Frameworks Under Evaluation

The two frameworks are both open-source and spec-driven, running on the same AI coding CLI. Both structure work through explicit specification and planning artifacts before implementation begins. They differ in how execution is organised: Spec-kit operates within a single agent conversation, while APM distributes work across multiple specialised agents.

1.2.1 GitHub Spec-kit

Spec-kit[1] is a specification-driven development toolkit developed by GitHub. It guides a single AI agent through a structured workflow using three sequential commands:

1. `/speckit-specify`: transforms a feature description into a structured specification containing requirements, user stories, and acceptance criteria.
2. `/speckit-plan`: converts the specification into a technical implementation plan with architecture decisions, data models, API contracts, and supporting documents.
3. `/speckit-tasks`: derives an executable task list (`tasks.md`) from the plan, with parallelism annotations and sequencing constraints.

Execution is triggered by a fourth command, `/speckit-implement`, which parses the task list and carries out each task in sequence. Additional commands support iterative refinement (`/speckit-clarify`), cross-artifact consistency checking (`/speckit-analyze`), and custom quality checklists (`/speckit-checklist`). A constitution mechanism, maintained via the `/speckit-constitution` command, allows users to establish architectural rules that the agent enforces throughout the workflow.

All planning and implementation happens within one continuous conversation. The agent acts as both planner and executor, and the full specification, plan, and task context accumulates in a single context window. Spec-kit produces a fixed set of artifact types (specification, plan, data model, contracts, research, tasks) regardless of project type; the templates are predefined and every project receives the same artifact layout.

[1] Documentation available at: https://github.github.com/spec-kit/

1.2.2 APM in Comparison

Where Spec-kit runs the entire workflow in a single conversation, APM separates planning, coordination, and execution across three agent roles, each operating in its own conversation with scoped context. Agents communicate through a file-based Message Bus; planning documents adapt to each project rather than conforming to a predetermined artifact layout; and the user acts as a message carrier between conversations. When an agent's context fills, a Handoff mechanism preserves working knowledge across conversations.

1.3 Testbed and Task Design

To evaluate the frameworks under realistic constraints, we used an existing open-source project, the `eclass-mcp-server`,[2] a Python-based Model Context Protocol (MCP)[3] server designed to interface with the `openeclass` platform.[4] Openeclass is an online platform for Greek universities that allows students to manage their courses and assignments.

1.3.1 Task Description

The task requires participants to implement two new announcement tools in the Python MCP server and to create a complete native C replica of the entire server with some explicit strict constraints.[5] The two tools are:

- `get_course_announcements`: retrieves announcements for a specific course.
- `get_general_announcements`: aggregates system-wide and per-course announcements.

The C implementation adds significant complexity: participants must manage code translation, memory safety, and protocol details across two different languages. The combination of feature implementation and full-language replication is designed to challenge the two frameworks and expose any differences between them, particularly when context window limits become a bottleneck.

The task admits more than one viable implementation path; a scoring rubric[6] awards points to any path that meets its per-test specifications.

1.3.2 Task Environment

Each session runs in a workspace prepared up-front. The working directory is a fresh clone of `eclass-mcp-server` checked out to a fixed patch-baseline commit against which the participant

[2]Repository available at: https://github.com/sdi2200262/eclass-mcp-server
[3]Protocol specification available at: https://modelcontextprotocol.io
[4]Repository available at: https://github.com/gunet/openeclass
[5]Task specification, participant package, and containerised testing environment available at: https://github.com/sdi2200262/bsc-apm-case-study-task
[6]Scoring rubric and grading infrastructure available at: https://github.com/sdi2200262/bsc-apm-case-study-infra

implements the task; the submission patch is the diff between this baseline and the participant's staged changes. A read-only reference clone of `openeclass` at a fixed inspection commit is also present, so any research the assistant performs into the legacy server's source reads from a fixed snapshot rather than a moving target. Alongside the workspace runs a containerised mock of the openeclass platform; the mock is built from that same inspection commit, only compiled, seeded with deterministic fixture data, and packaged in a Docker image, and paired with a deterministic mock single-sign-on service. The assistant therefore inspects the very source that the running mock executes.

The mock is the participant's integration target during development and the same target the grader replays when scoring, so every cohort participant develops and is graded against the same fixed state of the platform. The exact pins, the upstream release tag the image is built from, the seeded fixtures, and the mock authentication surface are documented in the Reference Environments subsection of the Evaluation Appendix.

1.4 Study Design

The study compares the two frameworks across a homogeneous cohort of participants drawn from University students and working professionals. The design is within-subjects: every participant contributes data points for both frameworks, so per-participant variance in skill, prior experience, and working style is absorbed into each pairwise comparison rather than confounding it.

1.4.1 Counterbalancing

Each participant completes the case study task twice, once with APM and once with Spec-kit, each session time-boxed to a maximum of three hours. Two factors are counterbalanced across the cohort: the framework order and the model assignment.

The framework-order factor varies which framework a participant encounters first. Counterbalancing the order lets any observed first-to-second-session improvement be attributed to task familiarity rather than to the framework itself; the resulting first-to-second-session learning is analysed in the Learning Effects subsection in Section 2. The model factor varies which Claude model drives both of a participant's sessions, held fixed within the participant. Each participant is assigned one of Claude Opus 4.7 and Claude Sonnet 4.6. The two models differ in capabilities and context-window capacity, one million tokens for Opus 4.7 against two hundred thousand for Sonnet 4.6.

The cohort is split across four assignment cells: APM first with Opus, Spec-kit first with Opus, APM first with Sonnet, and Spec-kit first with Sonnet. Holding the model fixed within participant lets the within-participant framework comparison absorb the model factor into the participant's own variance; the cohort-level analysis additionally compares the framework effect across the two models.

The two sessions are isolated from one another so that no code, workspace state, or assistant context from the first session can influence the second. The full session-isolation procedure is documented in the Submission Workflow subsection of the Evaluation Appendix.

1.4.2 Data Source

Each session produces a participant submission composed of three artefacts: a solution patch the participant writes against the patch baseline, the set of Claude Code JSONL transcripts the CLI persists for every conversation opened during the session, and a post-session questionnaire response.

The solution patch is the participant's functional contribution and the input to scoring. The transcripts are the source of every cost, time, message-activity, and reliability measurement: each transcript provides a structured record of every conversation, so the measurements are extracted programmatically rather than by manual annotation. Auxiliary records that Claude Code persists for any subagents the conversation dispatched and for any tool results it offloaded from the main file travel with the transcripts, so a complete record of the session is available for measurement.

The post-session questionnaire complements the transcript-derived measurements with self-reported workload and perception items and is submitted alongside the patch and the transcripts through a single per-session combined web form. Its captured items and response scale are documented in the Post-Session Questionnaire Instrument subsection of the Evaluation Appendix.

Participant data is anonymised and encrypted before storage; the sanitisation scheme is documented in the Data Privacy and Sanitisation subsection of the Evaluation Appendix.

1.5 Metrics

Every measurement defined in this section is extracted from the participant's submission by the same procedure across the cohort. The economic, process, and supplementary measurements are derived from the session's transcripts; the functional measurements are derived from the solution patch, scored against the rubric introduced earlier.

Every transcript-derived measurement is computed at two scopes per submission, per-session and per-phase, where the phases are the planning portion and the implementation portion of the session. A third per-role scope is emitted explicitly for APM submissions, splitting the session by Planner, Manager, and Worker.

Per-session is the headline scope on which the two frameworks are compared, since it places APM's multi-conversation workflow and Spec-kit's single conversation on the same denominator. Per-phase and per-role views locate where within a session each framework spends its budget and effort. Phase boundaries and APM roles are detected automatically; the detection rules are documented in the APM Role Detection and Framework Phase Detection subsections of the Evaluation Appendix. The functional dimension admits no phase or role decomposition and is reported per submission only.

Each measurement carries an anchor flag distinguishing Primary measures (those that drive a research question and enter hypothesis testing), Secondary measures (those that decompose or support a Primary), and Supplementary measures (descriptive only). The subsections below specify each measurement dimension one by one.

1.5.1 Economic

The primary economic measurement is the **dollar-equivalent cost**, computed by pricing every call recorded in the session's transcripts against Anthropic's published per-token rate card at the study's snapshot date, across the five token categories the rate card specifies. Calls made by the conversation's primary agent are priced at that conversation's declared model; calls made by subagents are priced against the model recorded in the platform's sibling subagent transcript for that dispatch.

A session's cost is reported as a point estimate when every dispatch resolves to a single model and as lower and upper bounds when a dispatch's model cannot be determined deterministically. The full attribution rules, the fallback resolution path, and the bracketing scheme are documented in the Per-Token Rate Card subsection of the Evaluation Appendix.

Each participant contributes one APM cost and one Spec-kit cost for the same task, compared pairwise across the cohort at the per-session scope.

1.5.2 Process

The process metrics capture the burden each framework imposes on the user across a session and are separated into how the user interacted with the model and how they invested their time during the session.

User-message activity is partitioned along two dimensions. The first is the kind of message the user typed:

- **turn prompts** are prompts submitted while the agent was idle, opening a new turn.Each turn is the wall-clock interval from a user prompt to the assistant's idle return.
- **steers** are interjections enqueued while the agent was still working, which the platform either delivers to the agent during the turn or abandons if the user retracts them before delivery. Steer activity is additionally reported as three counts: **steer attempted**, **steer delivered**, and **steer abandoned**.

Each prompt and each delivered steer is assigned to one of three buckets along the second dimension:

- **short** for brief acknowledgments or routing messages.
- **protocol** for framework-protocol messages such as APM or Spec-kit slash-command invocations.
- **standard** for substantive task-related prompts.

The **bucket-share distribution** reports the share of each bucket in the session's total user-message count, separately for prompts and for delivered steers and jointly across both; **magnitude per bucket** reports the mean character count of messages in each bucket. Together

these views expose whether a framework's user burden is concentrated in mechanical message-shuttling (high protocol share, low magnitude) or in substantive corrections and guidance (high standard share, high magnitude).

The temporal shape of the session is captured by a family of turn-time metrics:

- **session duration** is the wall-clock time from the first user message of the session to the last assistant message, bounded above by the three-hour cap.
- **turn count** is the number of turns recorded across the session's conversations.
- **mean turn duration** is the average turn-time, complemented by the **median**, the **ninety-fifth percentile**, and the **standard deviation** of the per-turn duration distribution.
- **turn time** is the wall-clock union of every turn's interval, reported in total and decomposed into the share spent in a single conversation at a time (**turn time solo**) and the share where two or more conversations were active simultaneously (**turn time concurrent**); for APM's multi-conversation design the decomposition surfaces whether the participant ran agents in parallel or serially.
- **inter-turn time** is the residual of the session duration once turn time has been subtracted: the wall-clock gap between consecutive turns, summed across the session. It is the assistant's idle time as recorded in the transcripts and is not interpreted as a measure of participant activity, since the participant may be reading, thinking, switching context, or away from the workstation during any gap.

Session duration, turn time with its solo and concurrent decomposition, and inter-turn time are reported at the per-session and per-phase scopes only; the wall-clock union and concurrency interpretations are not well-defined at role scope, where a role lives inside a single conversation file whose own turns cannot overlap.

1.5.3 Functional

The functional dimension scores each submission against the task's scoring rubric introduced earlier. The rubric is a fixed set of tests; each test checks one expected behaviour, produces a pass-or-fail verdict, and contributes a calibrated point weight to a 100-point total, so a submission that delivers only part of the task earns proportionally for what works.

The 100 points are split across the task's two implementations. The Python implementation carries 41 points: a baseline section that confirms the four pre-existing tools still work, and two feature sections covering the new course-announcement and general-announcement tools. The C replica carries 59 points: a build-and-stack section, a baseline section that re-implements the same four tools from scratch, the two announcement features mirrored against the C binary, and a memory-safety section exercised under Valgrind. The per-section point allocations, the per-test definitions, and the assertion logic each test uses are documented in the Test Suite Composition subsection of the Evaluation Appendix.

The primary functional outcomes are the total rubric score and the per-section scores: the first summarises the framework comparison in a single number, the second locates where each framework's work lands within the rubric.

1.5.4 Supplementary

A small set of supplementary descriptors is reported per framework alongside the primary and secondary measurements. They are descriptive only and do not enter the paired hypothesis testing.

The **tool success rate** is the proportion of every tool invocation result recorded on the session's main transcripts that does not carry an error flag, spanning every tool the assistant may invoke (file edits, shell, search, web fetches, MCP calls) without weighting by tool kind. Tool calls made inside subagent dispatches are not included. The rate is dominated by the underlying model's tool-use capability rather than by framework choice.

The **subagent dispatch count** is the number of `Task`-tool dispatches the main agent issued in scope, and the **delegated tool-use count** is the total number of tool invocations those dispatched subagents performed inside their own conversations. Together they characterise how much work each framework offloads to delegated subagents.

The **model-invoked Skill count** is the number of platform-rendered Skill bodies the agent itself caused to be injected through the Skill tool, surfacing self-invocation behaviour that does not appear in the user-message activity.

1.5.5 Analysis Plan

Each measurement is reported at the cohort level as a descriptive summary first (mean, standard deviation, median, and inter-quartile range per framework and per model band) and then as a paired framework comparison. The descriptive summaries report what a typical APM session and a typical Spec-kit session cost, take, and produce, and locate the spread of the cohort around those typical values. The paired comparison establishes whether the per-participant difference between the two frameworks is systematic across the cohort and how large that difference is relative to the per-participant variance.

The paired comparison uses a paired t-test on the per-participant difference. Given the two paired values x_i^{APM} and x_i^{SK} for participant i, the difference $d_i = x_i^{\mathrm{APM}} - x_i^{\mathrm{SK}}$ has mean $\bar{d}$ and standard deviation s_d across the cohort of n participants; the test statistic is

$$t = \frac{\bar{d}}{s_d/\sqrt{n}},$$

and the reported p-value comes from the two-sided t distribution on $n-1$ degrees of freedom.

The effect size is Cohen's d_z, the standardised mean difference in pooled within-pair-variance units,

$$d_z = \frac{\bar{d}}{s_d},$$

which expresses the framework gap in standard deviations rather than in the metric's native units and so makes effect sizes comparable across metrics of different scale.

Every primary measurement is analysed this way: total rubric score, per-section scores, per-session cost, phase-broken cost shares, bucket-share distribution and magnitude per bucket across turn prompts and delivered steers, the turn-time decompositions, and session duration. The model factor is handled by stratification: paired comparisons are reported within each model band and then jointly across the cohort, exposing any framework effect that holds in one band but not the other.

Correlations between economic, process, and functional metrics are reported as Pearson coefficients within each framework and across the combined cohort. Pearson's r is the standardised linear-association statistic on the pair (x_i, y_i):

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2 \, \sum_i (y_i - \bar{y})^2}},$$

taking values in $[-1, 1]$. Correlation magnitudes are interpreted alongside the within-framework spread; they surface whether framework-induced variation in cost or process burden translates into functional outcomes.

Learning effects are analysed separately across the counterbalanced orderings. A learning-effect matrix reports mean first-to-second-session improvement across both framework orderings for each metric; an order-effect comparison between APM-first and Spec-kit-first directions tests whether the order of exposure affects the improvement; and the aggregate first-to-second-session improvement across orderings gives the baseline task-learning estimate.

2 Results

2.1 Economic

[Forthcoming. Quantitative results for the per-session cost and the phase-broken decomposition. Tables: cohort-level descriptive summaries (mean, standard deviation, median, IQR) per framework and per model band; paired APM-versus-Spec-kit cost distributions stratified by model band and pooled across the cohort, with paired t-test statistics, p-values, and Cohen's d_z; planning-versus-implementation cost shares per framework; per-role cost shares across Planner, Manager, and Worker for the APM cohort; cost composition by token category and by main-versus-delegated population; Pearson correlations against rubric score within and across frameworks.]

2.2 Process

[Forthcoming. Quantitative results for the user-burden metrics. Tables: bucket-share distribution and magnitude per bucket across turn prompts and delivered steers, paired across frameworks with t-statistics and Cohen's d; steer attempted, delivered, and abandoned counts per framework; turn-time decompositions (solo, concurrent, inter-turn) and the per-turn duration distribution; session-duration distributions against the three-hour cap; Pearson correlations against rubric

score. Self-reported workload items from the post-session questionnaire are reported alongside the instrumented coordination metrics.]

2.3 Functional

[Forthcoming. Quantitative results for the 100-point rubric. Tables: total-score distribution per framework, per-section score distributions across the Python and C implementations, per-test pass rates, baseline-regression rates, and paired framework comparisons with t-statistics and Cohen's d. Perceived framework effectiveness from the post-session questionnaire is reported alongside the rubric outcomes.]

2.4 Supplementary

[Forthcoming. Descriptive per-framework summaries for the supplementary metrics: tool success rate, subagent dispatch count, delegated tool-use count, and model-invoked Skill count. Reported as cohort-level descriptives per framework and per model band without paired hypothesis testing.]

2.5 Learning Effects

[Forthcoming. Quantitative results for the learning-effect analysis: first-to-second-session improvement matrix across framework orderings for functional, economic, and process metrics; order-effect comparison between APM-first and Spec-kit-first directions; and the aggregate baseline task-learning estimate across orderings.]

3 Discussion

3.1 Economic Efficiency

[Forthcoming. Discussion of the per-session cost comparison, the phase-level decomposition, and the per-role decomposition for the APM cohort, read against the workflow structures of the two frameworks described in the Frameworks Under Evaluation subsection in Section 1. The discussion treats two cost-shaping mechanics that the per-session metric integrates over. The first is the choice of context-reset strategy within a session: platform autocompaction substitutes a summarised prior context for a price below a full handoff but discards fidelity in the process, while a manual handoff preserves continuity at a higher per-event cost. The second is the context-window capacity of the model driving the session: the cost dynamics of a single concentrated window against several lower-filled distributed windows are not symmetric across the one-million-token and two-hundred-thousand-token regimes. The cumulative cost split observed in the Results is read against these mechanics, separately within each capacity band, and the framework comparison is interpreted as the net of the strategy and capacity contributions; the per-role decomposition further locates which APM role carries the structural overhead within that comparison.]

3.2 Process Efficiency

[Forthcoming. Discussion of the user-burden comparison across turn-prompt and steer activity, the reliability of agent output, and the temporal shape of each session, paired with the self-reported workload items from the post-session questionnaire.]

3.3 Functional Efficacy

[Forthcoming. Discussion of the rubric-score comparison and the per-section pass-rate decomposition, framed against the design motivation for multi-agent coordination.]

3.4 Threats to Validity

This subsection enumerates the internal, external, and construct validity threats and the mitigations applied. Each threat points back to the specific subsection that motivates it.

3.4.1 Internal Validity

Learning effects from the within-subjects design. The within-subjects design declared in the Study Design subsection in Section 1 has each participant complete the same task twice, so task familiarity from the first session can influence performance in the second. Counterbalancing the framework order across participants, as described in the Counterbalancing subsection in Section 1, mitigates the framework-effect confound but does not eliminate the underlying learning curve. The Learning Effects analysis described in the Analysis Plan subsection in Section 1 quantifies the mean improvement between sessions across both orderings and is the primary defence against this threat.

Self-reported versus observed measures. The post-session questionnaire described in the Post-Session Questionnaire subsection in Section 1 pairs the instrumented, transcript-derived metrics with self-reported workload, confidence, and perceived framework effectiveness; observed divergences between self-report and instrument are reported as findings rather than absorbed as an unstated limitation. Residual risk remains: self-reports are subject to recall bias and social-desirability pressures that the instrument-side measurements cannot independently correct.

3.4.2 External Validity

Generalisation scope. The case study is run on a single AI coding CLI, against a complex software engineering task requiring two-language implementation and legacy-codebase research, described in the Task Description subsection in Section 1, in a setting positioned between laboratory and field conditions. Findings generalise within this scope and do not generalise to multi-platform settings, dynamic cross-model routing, simpler or less knowledge-intensive tasks, or pure laboratory or natural-setting conditions.

Temporal decay of findings. The AI coding assistant field is evolving rapidly. Effect sizes and descriptive claims throughout this chapter (rate-card prices, command sets, framework

releases, and model availability) are tied to the May 2026 snapshot stated in Section 1. The dollar-equivalent cost values reported under Section 2 rest on the rate-card snapshot pinned in the Per-Token Rate Card subsection of the Evaluation Appendix; results that rest on those specific rates are defensible against the snapshot but may require re-estimation under a materially different rate card.

3.4.3 Construct Validity

Behavioural input buckets as process-efficiency proxy. The short, protocol, and standard input buckets defined under the Process metric in the Metrics and Data Collection subsection in Section 1 are extracted from the transcript content for this study. They are not validated against an external coordination-burden instrument. Their validity as proxies for human coordination effort rests on face validity alone: the relative ordering of frameworks along these metrics is robust, but the absolute magnitude of differences is interpretive rather than calibrated.

Instrument coverage limits. The 100-point rubric, defined in the Test Suite Composition subsection of the Evaluation Appendix, captures correctness and robustness but not code quality, maintainability, security posture, or long-term technical debt. Metric extraction is further limited to the Claude Code JSONL transcripts and their sibling subagent files that accompany each participant submission, as declared in the Data Source subsection in Section 1: any interaction with the AI that took place outside the CLI's transcript surface is invisible to the metric set.

Replica versus live openeclass. The shared Docker-based replica of the openeclass platform described in the Task Environment subsection in Section 1 pins the platform to a fixed upstream baseline and seeds deterministic fixture data. The shared replica makes the cohort comparable and the grading reproducible, and removes any parity gap between what participants build against and what the evaluation runs against. The trade-off is that the task is performed against a replica rather than against any live university instance of openeclass: any property of a live instance that the replica does not reproduce, such as timezone handling of locale-formatted dates, session-cookie behaviour under load, cross-origin interactions with the mock CAS service, or upstream behaviours that have shifted since the baseline commit, is not exercised by the case study. Replica construction and seeding details are documented in the Baseline Pins and Reference Environments subsection of the Evaluation Appendix.